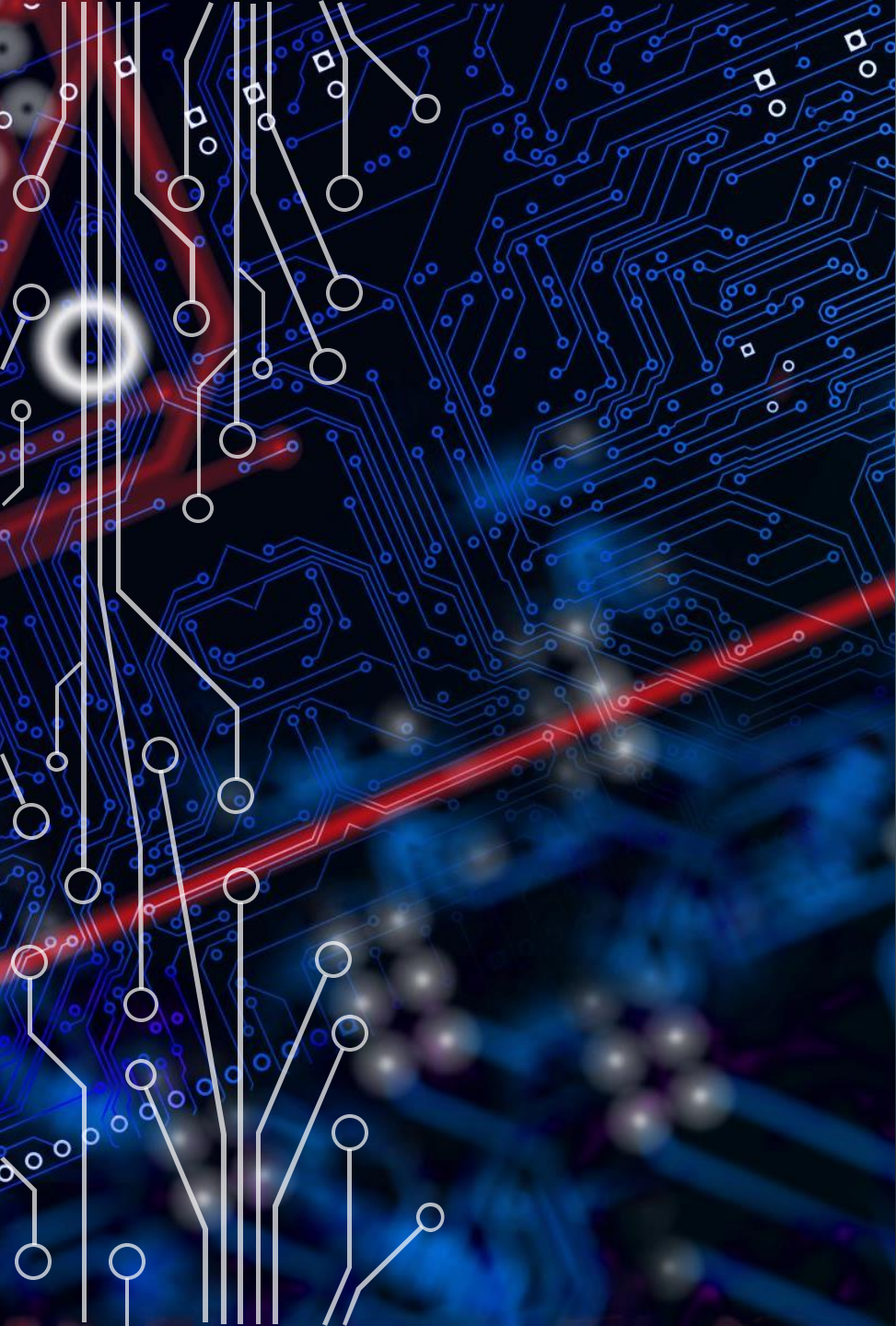
A decorative graphic consisting of thin, grey, stylized circuit lines with small circles at the ends, extending horizontally from the left and right sides of the central black box.

AIRDRAW SYSTEM: MOTION- CONTROLLED DRAWING ON PIC24

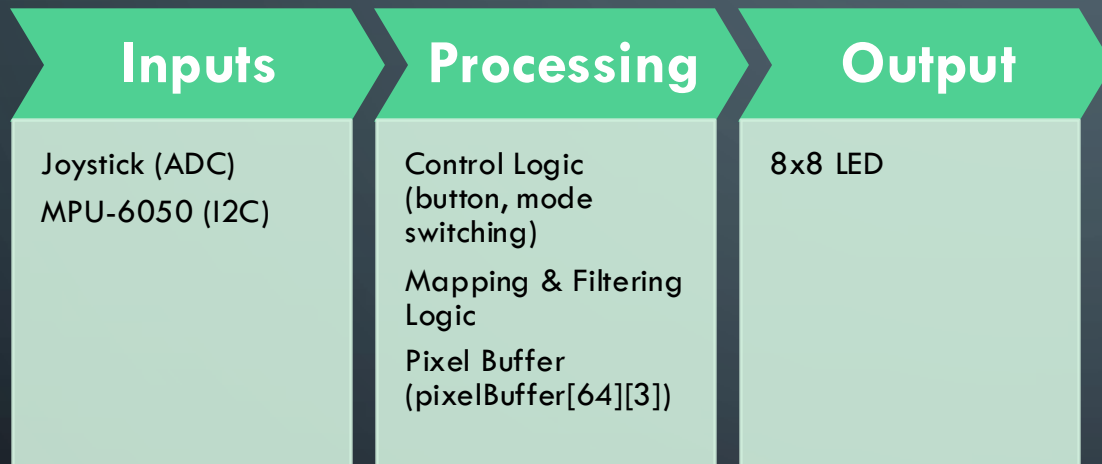
ALI ENDRIS, IDRES ALI, ABDIFATAH HASSAN, ADILLO SERO



PROJECT OVERVIEW

- **Purpose of the System:**
- Provide a motion-controlled drawing interface using a microcontroller
- Allow users to create drawings using real-time sensor input
- **Core Concept:**
- MPU-6050 tracks motion along x and y axes
- Motion data is filtered and mapped to an 8×8 LED matrix
- System converts physical movement into visual output
- **User Interaction:**
- Button toggles drawing mode (start/stop drawing)
- Joystick provides reset/clear functionality

SYSTEM ARCHITECTURE PART 1



- **How They Interact:**
- Inputs provide position or motion data
- Control logic determines system mode and behavior
- Mapping/filtering converts data into 8×8 grid coordinates
- Pixel buffer stores RGB values for each LED and being printed/updated in real time

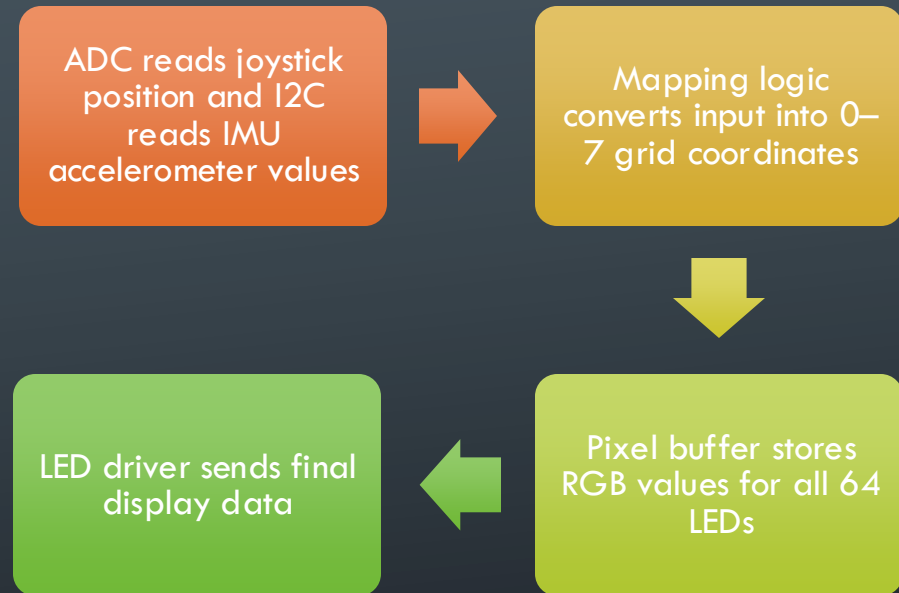
SYSTEM ARCHITECTURE PART 2

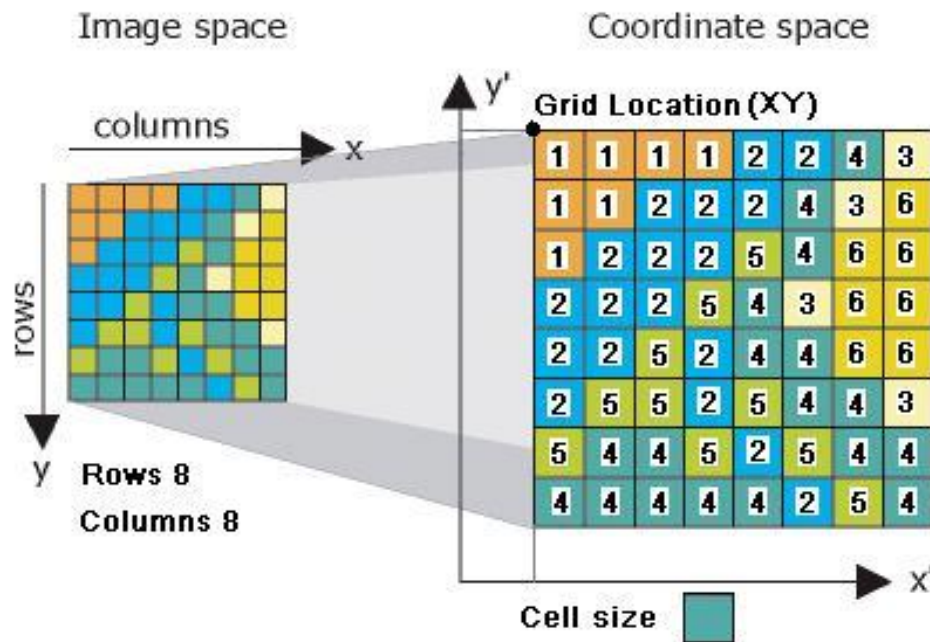
Important Logic in Our Code

- pixelBuffer[64][3]
 - Stores red, green, and blue values for each LED
 - Allows drawings to stay on the screen
- Deadzone algorithm
 - Ignores small joystick movement near the center
 - Prevents cursor drift
- IMU smoothing filter
 - Blends new accelerometer data with previous data
 - Reduces jitter before mapping to the LED grid

Click-count button logic

- Tracks single, double, and triple clicks
- Allows multiple controls using one joystick button





List of cell values

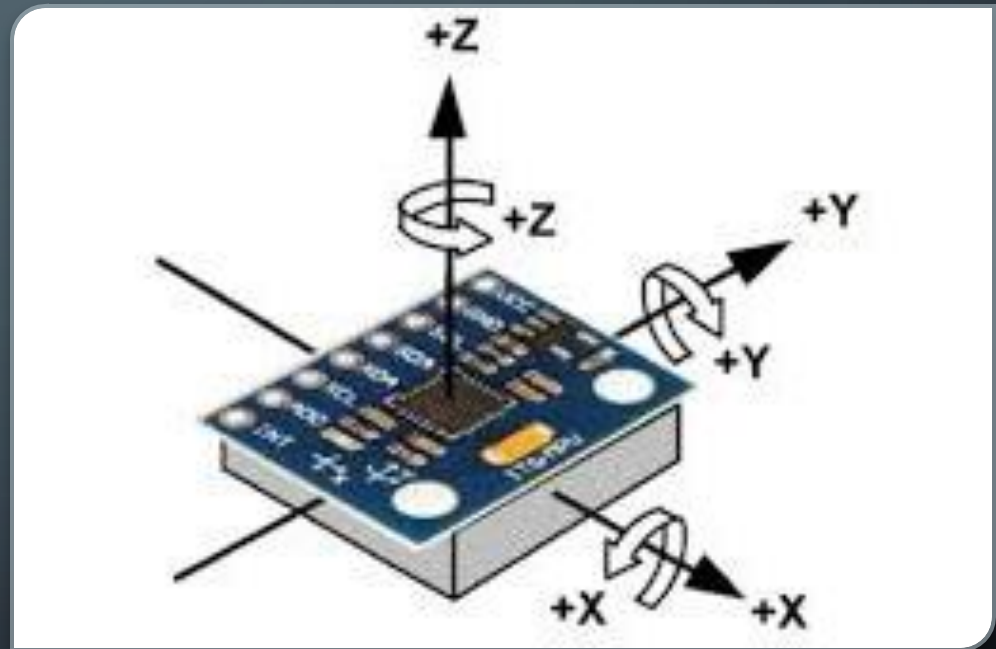
[111122431122243612225466222543662252446625525443544525444444254]

CONTRIBUTIONS-ALI ENDRIS

- **Motion-to-Cursor Mapping**
- Converted sensor data into x-y cursor movement
- Implemented sensitivity and calibration logic
- Handled boundary conditions (screen edges)

CONTRIBUTIONS-ADILLO SERO

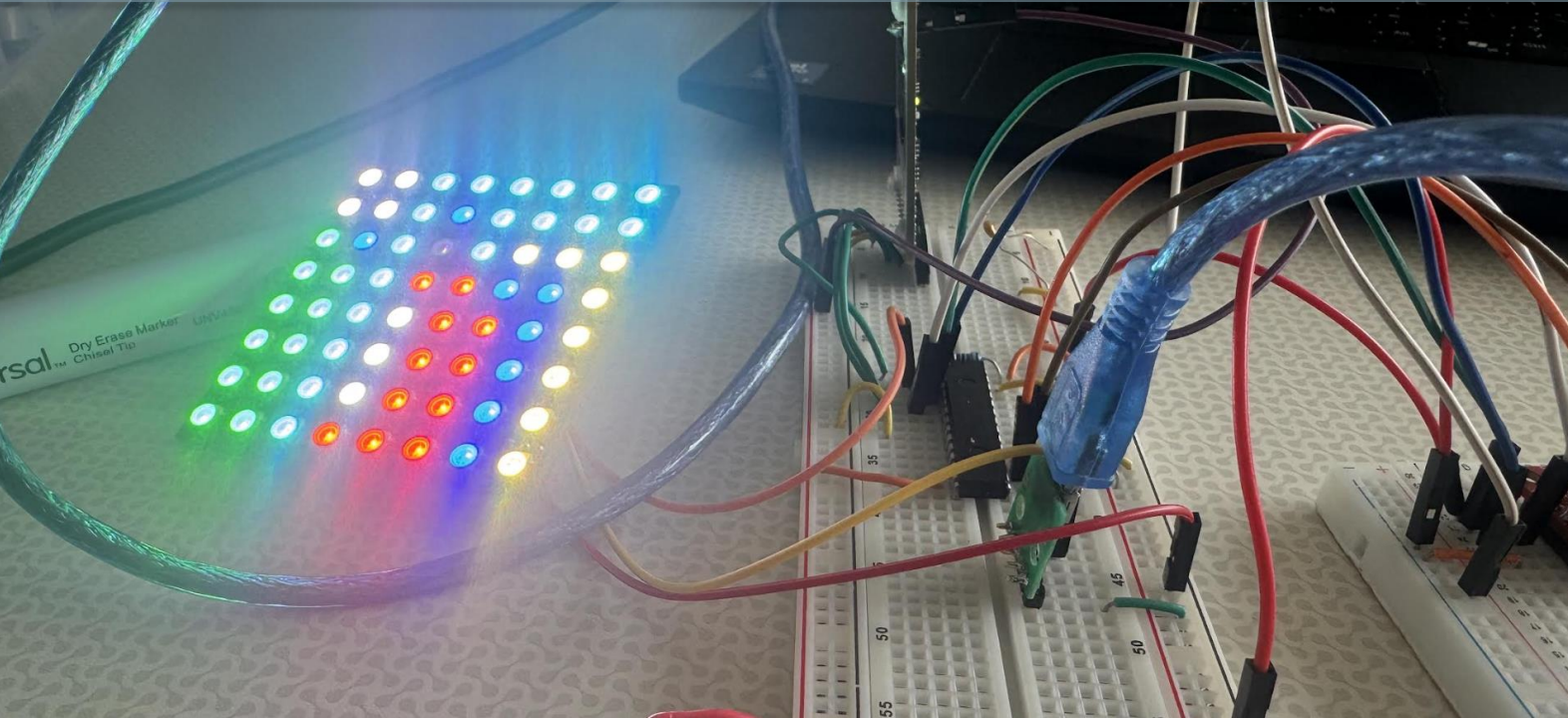
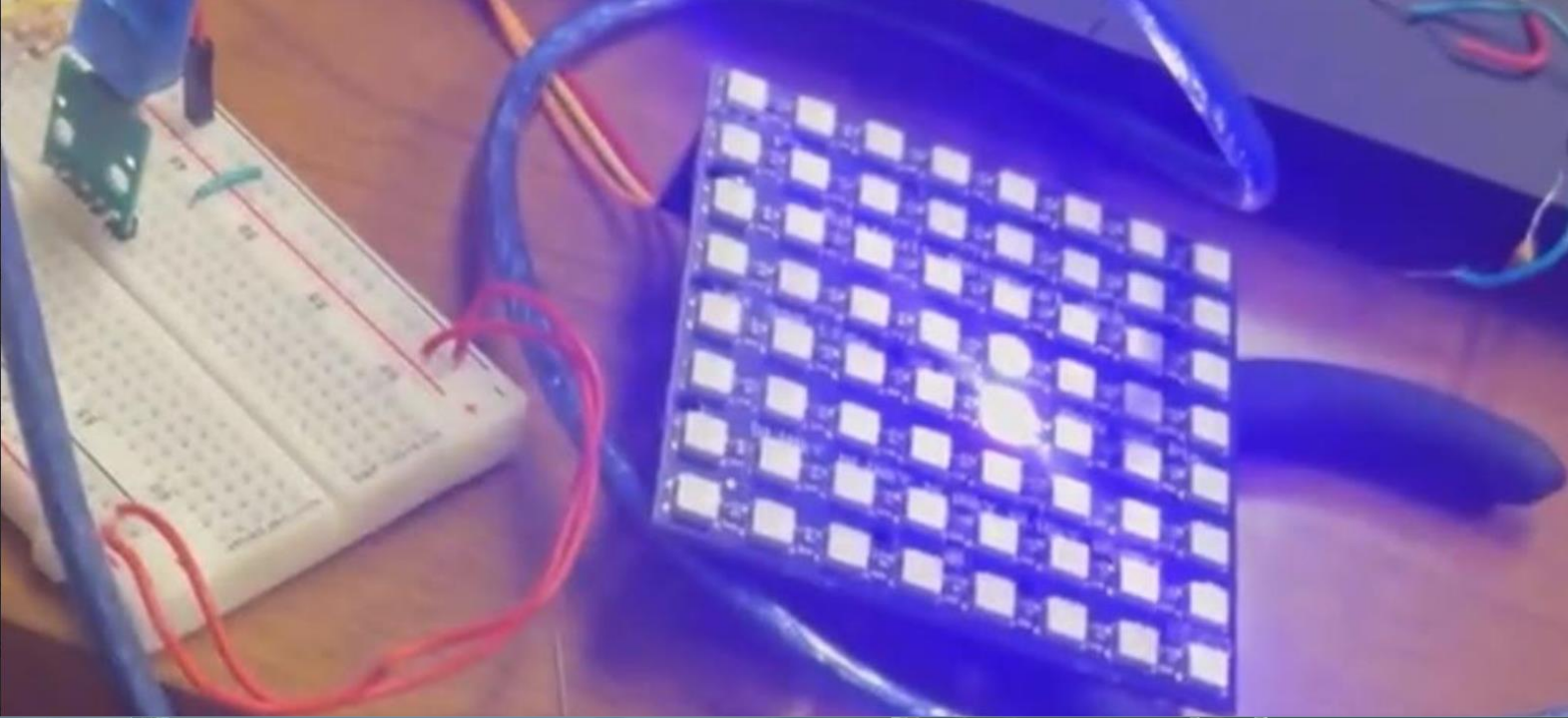
- **Sensor Integration & Data Processing**
- Implemented IMU(MPU-6050) communication using I2C
- Read raw accelerometer data
- Prepared filtered sensor data for mapping
- Output processed directional data for mapping



CONTRIBUTIONS-ABDIFATAH HASSAN

- **LED Display & Rendering**
- Interfaced with the LED matrix display
- Implemented pixel drawing logic
- Displayed cursor movement in real time
- Managed screen refresh and drawing persistence





CONTRIBUTIONS- IDRES ALI

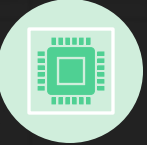
- Input Controls & System Integration
- Implemented button-driven drawing control (pen up/down)
- Designed multi-click detection for mode switching (color, reset)
- Implemented joystick deadzone for stable cursor control

API DESIGN- CUSTOM API/LIBRARY



read_adc(channel)

Reads analog voltage from the joystick (X/Y axes) and converts it into a digital value (0–1023) for position tracking.



send_pixel_byte(byte)

Sends one byte of color data to the LED matrix using precise timing, enabling control of individual LED colors.



get_air_coords(int *x, int *y)

Handles communication with the MPU-6050 over I2C, allowing the system to retrieve accelerometer data for motion-based control.



init_hardware()

Initializes all system components, including the ADC, I2C, and the IMU sensor, to prepare the system for operation.

How to Use This Library Again

Include header files (.h)

Call `init_hardware()` to initialize system

Use `read_adc()` or `get_air_coords()` for input

Update display using pixel functions



RESULTS AND SHORTFALLS

Shortfalls

IMU control less precise than desired

Loss of resolution (LED)

Error handling algorithm

Controls

QUESTIONS?